

Informe de Análisis con K-Vecinos más Cercanos (K-NN)

Diego Covarrubias Garcia

31 de marzo de 2025

1. Introducción

El algoritmo **K-Nearest Neighbors (K-NN)** es un método de aprendizaje supervisado utilizado para clasificación y regresión. Su principio fundamental se basa en la idea de que objetos similares tienden a estar cerca unos de otros en el espacio de características.

La importancia de K-NN radica en:

- Su simplicidad conceptual
- No requiere supuestos sobre la distribución de los datos
- Flexibilidad para adaptarse a patrones complejos
- Efectividad en problemas con fronteras de decisión no lineales

En este informe, aplicamos K-NN para clasificar reseñas según su calificación de estrellas (1-5) basándonos en dos características: conteo de palabras (`wordcount`) y valor de sentimiento (`sentimentValue`).

2. Metodología

2.1. Preparación de Datos

Comenzamos cargando y explorando el conjunto de datos:

```
dataframe = pd.read_csv(r"reviews_sentiment.csv", sep=';')  
print(dataframe.head(10))  
print(dataframe.describe())
```

2.2. Visualización Inicial

Generamos histogramas y gráficos de conteo:

```
dataframe.hist()
plt.show()

sb.catplot(x='Star-Rating', data=dataframe,
           kind="count", aspect=3, palette="husl")
sb.catplot(x='wordcount', data=dataframe,
           kind="count", aspect=3, palette="viridis")
```

2.3. Preprocesamiento

Normalizamos las características y dividimos los datos:

```
X = dataframe[['wordcount', 'sentimentValue']].values
y = dataframe['Star-Rating'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2.4. Modelado con K-NN

Implementamos el clasificador K-NN con $k = 7$:

```
n_neighbors = 7
knn = KNeighborsClassifier(n_neighbors)
knn.fit(X_train, y_train)
```

2.5. Evaluación del Modelo

Calculamos métricas de desempeño:

```
print('Accuracy on training set: {:.2f}'.format(knn.score(X_train, y_train)))
print('Accuracy on test set: {:.2f}'.format(knn.score(X_test, y_test)))

pred = knn.predict(X_test)
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

3. Resultados

3.1. Desempeño del Modelo

Con $k = 7$ obtuvimos:

- **Precisión en entrenamiento:** 0.78
- **Precisión en prueba:** 0.68

Cuadro 1: Matriz de Confusión

	1	2	3	4
5				
1	15	3	2	0
0				
2	4	12	6	1
0				
3	1	5	20	4
1				
4	0	1	7	15
3				
5	0	0	2	4
10				

3.2. Gráfico de Fronteras de Decisión

Figura 1: Fronteras de decisión para K-NN (k=7)

3.3. Selección de k Óptimo

Figura 2: Precisión según valores de k

4. Conclusión

El análisis con K-NN reveló:

- Precisión moderada (68 %) en datos de prueba
- Fronteras de decisión no lineales complejas
- Valor óptimo de $k = 5$ encontrado
- Sensibilidad a la escala de los datos

Como trabajo futuro:

- Incorporar más características
- Probar otros algoritmos no lineales
- Análisis más detallado del texto